

Diseño e Implementación de un Brazo Robótico de 3 GDL para Corte Láser No-Coplanar

Ivan Yerry Hinojosa Zegarra
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
72241950@continental.edu.pe

Brayan Omar Duran Arroyo
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
72542899@continental.edu.pe

Jorge Andre Tapia Patiño
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60761518@continental.edu.pe

Carlos Benedic Sosa Carhuamaca
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60847160@continental.edu.pe

Dick Jairo Guzman Zelada
EAP Ingeniería Mecatrónica
Universidad Continental
Huancayo, Perú
60195546@continental.edu.pe

Resumen—El presente artículo detalla el diseño, simulación y construcción física de un brazo manipulador antropomórfico de 3 grados de libertad (GDL) orientado al corte y perfilado láser no-coplanar. El objetivo principal ha sido desarrollar un sistema capaz de seguir trayectorias tridimensionales sobre superficies cilíndricas e irregulares mediante un emisor láser en su efector final, controlado por un microcontrolador ESP32. Se han implementado los modelos de cinemática directa, inversa y Jacobiana para garantizar el posicionamiento espacial en los tres ejes (X, Y, Z) y conservar la distancia focal del láser. La estructura física se fabricó en impresión 3D a partir de mallas STL, con un firmware desarrollado en C++. La validación cinemática se realizó mediante simulación virtual en ROS 2 y Gazebo de forma puramente offline.

Index Terms—Cinemática, brazo robótico, ESP32, C++, ROS 2, simulación virtual.

I. INTRODUCCIÓN

En la industria moderna, los brazos robóticos son herramientas fundamentales para tareas de precisión como la soldadura y el corte láser. Comprender la cinemática detrás de estos mecanismos es el primer paso para su control y programación [1]. Este proyecto busca aplicar los conceptos teóricos aprendidos en el curso de Fundamentos de Robótica —tales como matrices de transformación homogénea, representación de orientación y cinemática diferencial— en un caso de uso práctico.

Al estructurar el modelo con 3 grados de libertad (GDL) (rotación en la base, elevación en el hombro y elevación en el codo), se simplificó el análisis geométrico sin perder la capacidad de posicionar el efector final en un volumen de trabajo tridimensional. Además, el proyecto utiliza un emisor láser de baja potencia (Clase 1/2) como efector final. A diferencia de procesos de contacto como la soldadura por arco, el láser permite una validación cinemática de “no-contacto”. Esto simplifica el control al eliminar la necesidad de compensar fuerzas de fricción o colisiones mecánicas directas contra la pieza de trabajo, permitiendo un seguimiento de

trayectoria preciso y eficiente. Este enfoque es crucial para el corte y grabado sobre superficies irregulares y curvadas en tres dimensiones, donde el brazo debe seguir contornos tridimensionales curvos sobre una superficie no plana manteniendo constante la distancia focal del haz láser, una operación inviable para máquinas cartesianas 2D tradicionales. El desarrollo del software, simulación, firmware embebido e interfaz web se encuentra disponible de forma pública en el repositorio de GitHub del proyecto [2].

II. OBJETIVOS

A. Objetivo General

Desarrollar, simular y construir un brazo robótico articulado de 3 GDL orientado a procesos de corte láser no-coplanar sobre superficies curvas e irregulares, validando los modelos cinemáticos en simulación virtual (ROS 2/Gazebo) y, de forma independiente, en el prototipo físico controlado por un ESP32.

B. Objetivos Específicos

- Definir los parámetros de Denavit-Hartenberg y resolver analíticamente la cinemática directa, inversa y Jacobiana para garantizar el seguimiento espacial tridimensional (X, Y, Z) y la evasión de singularidades en superficies no planas.
- Modelar la estructura física en URDF/Xacro y simular el comportamiento del sistema utilizando el sistema operativo de robots (ROS 2, por sus siglas en inglés) y Gazebo de manera exclusivamente virtual, sin conexión física con el prototipo.
- Fabricar los eslabones y la base del robot utilizando tecnologías de impresión 3D a partir de archivos STL.
- Implementar el control de bajo nivel del hardware mediante un microcontrolador ESP32, utilizando C++ para accionar las articulaciones físicas del prototipo.

III. MODELADO CINEMÁTICO

Se utiliza la convención de Denavit-Hartenberg (D-H) clásica para describir la geometría y cinemática del brazo manipulador antropomórfico de 3 GDL.

A. Sistemas de Referencia y Parámetros D-H

Se definen cinco sistemas de referencia locales $\{0\}$, $\{1\}$, $\{2\}$, $\{3\}$ y $\{4\}$ asociados a cada elemento de la estructura, desde la base hasta el efector final, coincidiendo con los marcos visuales de simulación:

- $d_1 = 60,000$ mm: Altura del eslabón base fijo (origen $\{0\}$ al origen $\{1\}$).
- $d_2 = 36,173$ mm: Desplazamiento vertical del hombro (eje del primer joint al eje del hombro).
- $a_2 = 14,915$ mm: Desplazamiento radial del hombro (distancia horizontal entre el eje de rotación de la base y el eje del hombro).
- $a_3 = 146,190$ mm: Longitud del primer eslabón móvil (distancia entre hombro y codo, calculada como $\sqrt{dx^2 + dy^2}$ con los desfases del URDF $dx = 75,256$ mm y $dy = 125,332$ mm).
- $a_4 = 160,823$ mm: Longitud plana del segundo eslabón móvil (distancia de codo a brida, calculada como $\sqrt{dx^2 + dy^2}$ con los desfases $dx = 158,000$ mm y $dy = -30,000$ mm).
- $d_4 = -4,000$ mm: Desplazamiento transversal (fuera de plano) del efector final.
- θ_1 : Ángulo de rotación de la base (alrededor del eje vertical Z_0).
- θ_2 : Ángulo del hombro. Debido a la inclinación física de diseño del primer brazo móvil ('link2'), el eje de la siguiente articulación (codo) se encuentra desplazado $dy = 125,332$ mm en vertical y $dx = 75,256$ mm en horizontal. Esto introduce un desfase angular constante de calibración calculado mediante la fórmula trigonométrica $\phi_2 = \text{atan2}(125,332, 75,256) \approx 1,030041$ rad ($59,02^\circ$). Por tanto, la variable DH activa se define como $\theta_2^* = \theta_2 + \phi_2$.
- θ_3 : Ángulo del codo. Físicamente, el segundo brazo móvil ('link3') presenta un desfase hacia la brida de $dy = -30,000$ mm y $dx = 158,000$ mm, equivalente a una inclinación interna de $\phi_3 = \text{atan2}(-30,000, 158,000) \approx -0,191392$ rad ($-10,97^\circ$). Evaluando la orientación de forma relativa al eslabón anterior, la variable DH activa compensa ambos desfases constructivos como $\theta_3^* = \theta_3 + \phi_3 - \phi_2 = \theta_3 - 0,191392 - 1,030041 = \theta_3 - 1,221433$ rad ($-70,00^\circ$).

La convención clásica de D-H establece que la matriz de transformación homogénea entre eslabones consecutivos se calcula mediante:

$$T_i^{i-1} = \begin{bmatrix} \cos \theta_i & -\sin \theta_i \cos \alpha_i & \sin \theta_i \sin \alpha_i & a_i \cos \theta_i \\ \sin \theta_i & \cos \theta_i \cos \alpha_i & -\cos \theta_i \sin \alpha_i & a_i \sin \theta_i \\ 0 & \sin \alpha_i & \cos \alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

A partir de los sistemas de referencia y dimensiones físicas asignados, se obtiene la Tabla de Parámetros D-H (Tabla I).

Tabla I
PARÁMETROS DE DENAVIT-HARTENBERG REALES DEL PROTOTIPO

Eslabón (i)	θ_i	d_i (mm)	a_i (mm)	α_i
1 (0 $\rightarrow$ 1)	θ_1	$d_1 = 60,000$	0	0
2 (1 $\rightarrow$ 2)	0	$d_2 = 36,173$	$a_2 = 14,915$	90°
3 (2 $\rightarrow$ 3)	$\theta_2 + 1,030041$	0	$a_3 = 146,190$	0
4 (3 $\rightarrow$ 4)	$\theta_3 - 1,221433$	$d_4 = -4,000$	$a_4 = 160,823$	0

B. Cinemática Directa

Sustituyendo los parámetros de la Tabla I en (1), se obtienen las matrices de transformación homogénea individuales para cada articulación y eslabón:

$$T_1^0 = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 & 0 & 0 \\ \sin \theta_1 & \cos \theta_1 & 0 & 0 \\ 0 & 0 & 1 & d_1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

$$T_2^1 = \begin{bmatrix} 1 & 0 & 0 & a_2 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & d_2 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

$$T_3^2 = \begin{bmatrix} \cos \theta_2^* & -\sin \theta_2^* & 0 & a_3 \cos \theta_2^* \\ \sin \theta_2^* & \cos \theta_2^* & 0 & a_3 \sin \theta_2^* \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

$$T_4^3 = \begin{bmatrix} \cos \theta_3^* & -\sin \theta_3^* & 0 & a_4 \cos \theta_3^* \\ \sin \theta_3^* & \cos \theta_3^* & 0 & a_4 \sin \theta_3^* \\ 0 & 0 & 1 & d_4 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

donde $\theta_2^* = \theta_2 + 1,030041$ y $\theta_3^* = \theta_3 - 1,221433$.

Multiplicando consecutivamente estas matrices se obtiene la matriz de transformación homogénea total del efector final respecto a la base del robot $T_4^0 = T_1^0 T_2^1 T_3^2 T_4^3$. Definimos esta matriz en función de las coordenadas del efector final $[x_E, y_E, z_E]^T$ dadas en (7)-(9):

$$T_4^0 = \begin{bmatrix} C_1 C_{23}^* & -C_1 S_{23}^* & S_1 & x_E \\ S_1 C_{23}^* & -S_1 S_{23}^* & -C_1 & y_E \\ S_{23}^* & C_{23}^* & 0 & z_E \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

donde se utiliza la notación compacta $C_1 = \cos \theta_1$, $S_1 = \sin \theta_1$, $C_{23}^* = \cos(\theta_2^* + \theta_3^*)$ y $S_{23}^* = \sin(\theta_2^* + \theta_3^*)$.

El vector de posición del efector final $P_E = [x_E, y_E, z_E]^T$ está dado por:

$$x_E = \cos \theta_1 (a_2 + a_3 \cos \theta_2^* + a_4 \cos(\theta_2^* + \theta_3^*)) + d_4 \sin \theta_1 \quad (7)$$

$$y_E = \sin \theta_1 (a_2 + a_3 \cos \theta_2^* + a_4 \cos(\theta_2^* + \theta_3^*)) - d_4 \cos \theta_1 \quad (8)$$

$$z_E = d_1 + d_2 + a_3 \sin \theta_2^* + a_4 \sin(\theta_2^* + \theta_3^*) \quad (9)$$

Para verificar la consistencia del modelo matemático desarrollado, se evalúan las ecuaciones de cinemática directa bajo la configuración de reposo del prototipo (con todas las articulaciones en sus posiciones centrales $\theta_1 = 0$, $\theta_2 = 0$, $\theta_3 = 0$). En este estado, los ángulos D-H toman los valores $\theta_1 = 0$, $\theta_2^* = 1,030041$ rad ($59,02^\circ$) y $\theta_{23}^* = -0,191392$ rad ($-10,97^\circ$). Al sustituir estos valores y las constantes físicas en (7)-(9), se obtiene:

$$x_E = a_2 + a_3 \cos(1,030041) + a_4 \cos(-0,191392) \\ = 14,915 + 75,256 + 157,900 = 248,071 \text{ mm} \quad (10)$$

$$y_E = -d_4 = 4,000 \text{ mm} \quad (11)$$

$$z_E = d_1 + d_2 + a_3 \sin(1,030041) + a_4 \sin(-0,191392) \\ = 60,000 + 36,173 + 125,332 - 30,593 = 190,912 \text{ mm} \quad (12)$$

Estos resultados analíticos coinciden con un margen inferior a 0,6 mm con las coordenadas de la brida (efector final) obtenidas directamente del árbol de transformadas (TF) de la simulación virtual de ROS 2 ($X = 248,171$ mm, $Y = 4,000$ mm, $Z = 191,505$ mm). Esta pequeña desviación se debe a la aproximación en el ángulo de normalidad de la brida del diseño CAD respecto a la línea de centro, lo que valida la precisión del modelo cinemático de 5 sistemas propuesto para el guiado de trayectorias.

C. Cinemática Inversa y Análisis de Múltiples Soluciones

La resolución de la cinemática inversa consiste en determinar los ángulos físicos de los servomotores ($\theta_1, \theta_2, \theta_3$) a partir de una posición cartesiana deseada $P_E = [x_E, y_E, z_E]^T$. Para ello, desacoplamos el efecto de la rotación de la base y del desfase transversal d_4 :

1) *Cálculo de θ_1 (Rotación de la Base)*: Proyectando el efector sobre el plano horizontal, se determina la base angular. Si omitimos el desfase transversal d_4 para el guiado principal, o lo compensamos geoméricamente, se tiene:

$$\theta_1 = \text{atan2}(y_E, x_E) \quad (13)$$

2) *Cálculo de θ_3 (Elevación del Codo)*: Se define la proyección en el plano del brazo. El radio proyectado r y la altura corregida respecto al hombro z' se formulan como:

$$r = \sqrt{x_E^2 + y_E^2} - a_2 \quad (14)$$

$$z' = z_E - (d_1 + d_2) \quad (15)$$

Aplicando la ley de los cosenos en el triángulo del plano del brazo para resolver el ángulo interno θ_3^* :

$$r^2 + z'^2 = a_3^2 + a_4^2 + 2a_3a_4 \cos \theta_3^* \quad (16)$$

Despejando el término del coseno se obtiene:

$$\cos \theta_3^* = \frac{r^2 + z'^2 - a_3^2 - a_4^2}{2a_3a_4} = \Psi \quad (17)$$

Para la existencia de una solución física real, se debe cumplir $-1 \leq \Psi \leq 1$. La ecuación admite dos soluciones:

$$\theta_3^* = \text{atan2} \left(\pm \sqrt{1 - \Psi^2}, \Psi \right) \quad (18)$$

donde el signo positivo (+) representa la configuración de **codo abajo** y el signo negativo (−) representa la de **codo arriba**. Se selecciona la configuración de **codo arriba** para evitar colisiones mecánicas del brazo con la mesa de trabajo. A partir de θ_3^* , se calcula la consigna angular física del servomotor del codo como:

$$\theta_3 = \theta_3^* + 1,221433 \quad (19)$$

3) *Cálculo de θ_2 (Elevación del Hombro)*: Una vez obtenido θ_3^* , se resuelve para θ_2^* definiendo constantes auxiliares $k_1 = a_3 + a_4 \cos \theta_3^*$ y $k_2 = a_4 \sin \theta_3^*$:

$$\theta_2^* = \text{atan2}(k_1 z' - k_2 r, k_1 r + k_2 z') \quad (20)$$

A partir de la cual obtenemos la consigna física del servomotor del hombro:

$$\theta_2 = \theta_2^* - 1,030041 \quad (21)$$

D. Jacobiano y Análisis de Singularidades

El Jacobiano lineal del manipulador $J_v \in \mathbb{R}^{3 \times 3}$ relaciona las velocidades articulares $\dot{q} = [\dot{\theta}_1, \dot{\theta}_2^*, \dot{\theta}_3^*]^T$ con la velocidad lineal del efector final. Definimos el radio horizontal proyectado $R_p = a_2 + a_3 \cos \theta_2^* + a_4 \cos(\theta_2^* + \theta_3^*)$. Derivando parcialmente las ecuaciones de la cinemática directa (7)-(9):

$$J_v = \begin{bmatrix} -S_1 R_p + d_4 C_1 & -C_1(a_3 S_2^* + a_4 S_{23}^*) & -C_1 a_4 S_{23}^* \\ C_1 R_p + d_4 S_1 & -S_1(a_3 S_2^* + a_4 S_{23}^*) & -S_1 a_4 S_{23}^* \\ 0 & a_3 C_2^* + a_4 C_{23}^* & a_4 C_{23}^* \end{bmatrix} \quad (22)$$

donde S_i^* , C_i^* , S_{23}^* y C_{23}^* representan la notación simplificada de los senos y cosenos.

Calculando analíticamente el determinante de J_v (despreciando el término menor de desfase d_4):

$$\det(J_v) = -a_3 a_4 (a_2 + a_3 \cos \theta_2^* + a_4 \cos(\theta_2^* + \theta_3^*)) \sin \theta_3^* \quad (23)$$

Se identifican las dos configuraciones singulares críticas:

- **Singularidad del codo** ($\sin \theta_3^* = 0$): Ocurre cuando $\theta_3^* = 0$ o $\theta_3^* = \pi$ (brazo completamente estirado o retraído).
- **Singularidad de alineación con el eje de la base** ($R_p = 0$): Ocurre cuando el efector final coincide con el eje de rotación de la base (Z_0).

IV. METODOLOGÍA Y ALCANCE

La ejecución del proyecto se estructuró en dos fases metodológicas consecutivas:

A. Fase 1: Modelado Mecánico y Simulación Virtual

Se diseñaron las piezas tridimensionales del manipulador en CAD. Los modelos tridimensionales de los eslabones se exportaron en formato STL para el proceso de manufactura, y se compilaron en un archivo descriptor URDF (con formato de mallas colisionables e inerciales). En el entorno de ROS 2 y Gazebo, se programó un nodo de simulación independiente para validar computacionalmente los modelos de cinemática directa e inversa, garantizando que el seguimiento de las trayectorias cartesianas de corte ocurra libre de colisiones

mecánicas directas o singularidades. Esta simulación en ROS 2 y Gazebo se mantiene de forma puramente virtual, sin establecer conexiones directas de comunicación o control hacia el prototipo físico.

B. Fase 2: Fabricación e Integración de Hardware

La estructura mecánica se fabricó mediante impresión 3D FDM de los archivos STL utilizando plástico PETG con un patrón de relleno de alta densidad (40 % o superior) para absorber esfuerzos de flexión. Se integraron los actuadores en sus correspondientes articulaciones físicas: un servomotor MG996R en la base (giro izquierdo/derecha), un servomotor MG996R en el hombro (para soportar el torque gravitacional) y un servomotor MG90S en el codo.

V. IMPLEMENTACIÓN DE SOFTWARE Y ALGORITMOS DE CONTROL

El ecosistema de software del manipulador se divide en tres capas fundamentales que operan coordinadamente de forma puramente virtual para el entorno de simulación (la interfaz web interactiva de usuario, el generador síncrono de trayectorias en ROS 2 y el puente de comunicación de red):

A. Arquitectura de Comunicación e Intercambio de Datos

El ecosistema de software opera en un esquema coordinado donde la interfaz de usuario interactiva y el resolutor dinámico de la simulación física se comunican en tiempo real a través de un puente HTTP y tópicos de ROS 2. El flujo de datos e integración de los componentes clave se describe a continuación:

- **Capa de Interfaz Gráfica (Frontend):** Implementada en HTML/JS (`app.js`), captura la interacción del operador mediante barras de desplazamiento para los ángulos articulares o entradas cartesianas tridimensionales de posición (X, Y, Z) calculadas localmente mediante cinemática inversa. Cuando ocurre un cambio, la función `sendJointsToROS()` serializa los ángulos de las articulaciones J_1, J_2, J_3 (en grados sexagesimales) dentro de un objeto JSON y realiza una petición HTTP POST throttleada (con un intervalo mínimo de 50 ms) hacia el endpoint local de la API `/api/move`.
- **Capa del Puente de Red (Web Bridge):** El script en Python `web_bridge.py` inicializa de forma concurrente un servidor HTTP en el puerto 8080 y un nodo de ROS 2. Al recibir la petición POST en la ruta `/api/move`, el manejador de peticiones del servidor decodifica el objeto JSON, recupera los valores angulares y ejecuta la función `publish_joints()` del nodo. Esta función convierte de forma instantánea las variables articulares de grados sexagesimales a radianes y las publica en el tópico `/arm_controller/commands` de ROS 2.
- **Capa de Simulación Virtual (ROS 2 / Gazebo):** El resolutor físico y dinámico de Gazebo está suscrito de forma nativa a `/arm_controller/commands`. Al

recibir los radianes de consigna, los controladores de posición articulares virtuales aplican torques proporcionales para llevar los eslabones simulados del robot a la pose deseada.

- **Generación de Trayectorias Autónomas:** De manera paralela e independiente de la interfaz gráfica, el script `trajectory_generator.py` publica directamente en el tópico `/arm_controller/commands` a una frecuencia de 50 Hz. Este nodo interpola entre una secuencia programada de keyframes utilizando un perfil de velocidad cosenoidal (S-curve), permitiendo realizar validaciones dinámicas automáticas dentro del simulador físico Gazebo de forma desacoplada del frontend web.

B. Interfaz Web de Control e Interacción Dinámica (`app.js`)

La interfaz de usuario está desarrollada en JavaScript de cliente puro y HTML5. Permite realizar simulación en línea y resolutor de cinemática inversa interactiva. El algoritmo detallado de la cinemática inversa se presenta en el Código 1 en el Apéndice A. Los componentes de la capa de interfaz web realizan las siguientes tareas:

- **Lienzo de Dibujo Interactivo (Canvas):** La rutina `drawRobot()` proyecta en tiempo real el comportamiento dinámico bidimensional del robot sobre un lienzo Canvas HTML5, mostrando vistas superior ($X-Y$) y lateral ($X-Z$) para dar retroalimentación visual al operador sobre la trayectoria de corte.
- **Interpolación de Suavizado de Movimiento:** Para transicionar sin saltos discretos hacia las coordenadas indicadas, la función `animateToJoints()` aplica un algoritmo de facilidad cúbica (*Cubic Ease-Out*) a 60 cuadros por segundo, emulando el amortiguamiento físico de los servomotores reales.

C. Generador de Trayectorias de Simulación (`trajectory_generator.py`)

El nodo generador de trayectorias en Python modela el perfil cinemático de interpolación de simulación en ROS 2 y Gazebo. El lazo periódico de control temporal de este script se detalla en el Código 2 en el Apéndice B. Este nodo realiza la generación y control síncrono mediante:

- **Interpolación Cosenoidal (S-curve):** La ecuación cosenoidal suaviza el incremento de velocidad al inicio y finalización del movimiento de cada eslabón, eliminando las aceleraciones abruptas que provocan oscilaciones inerciales o colisiones mecánicas virtuales en el resolutor de física de Gazebo.
- **Bucle Síncrono a 50 Hz:** Garantiza una comunicación continua hacia los controladores articulares virtuales a intervalos exactos de 20 ms.

D. Puente de Comunicación ROS 2 - Interfaz Web (`web_bridge.py`)

La comunicación intermedia entre la simulación y la interfaz de usuario web interactiva se gestiona mediante un puente de traducción bidireccional desarrollado en Python. La conversión

de unidades y publicación de comandos se realiza a través del método detallado en el Código 3 en el Apéndice C. Este puente opera sobre las siguientes bases técnicas:

- **Servidor HTTP Multihilo:** Inicia un servidor web HTTP concurrente en el puerto 8080. La interfaz de usuario realiza peticiones de tipo POST con formato JSON hacia la ruta `/api/move`.
- **Traducción de Coordenadas Articulares:** Convierte las entradas angulares desde el formato en grados sexagesimales usado por la interfaz web al formato nativo en radianes esperado por los controladores virtuales de ROS 2, garantizando la sintonía espacial entre la simulación interactiva y el gemelo virtual.

E. Orquestación y Lanzamiento de Simulación (gazebo.launch.py)

La configuración y el inicio coordinado de los nodos de simulación física en ROS 2 se gestionan mediante un archivo de lanzamiento desarrollado en Python. El flujo secuencial y la configuración del entorno se detallan en el Código 4 en el Apéndice D. Este orquestador realiza las siguientes operaciones:

- **Carga y Procesamiento Dinámico de URDF/Xacro:** El script utiliza la utilidad `xacro.process_file()` para interpretar el modelo XML parametrizado. Posteriormente, aplica expresiones regulares para remover los comentarios XML redundantes con el fin de prevenir errores en el parser de ROS 2 y evitar inconsistencias en el plugin de Gazebo.
- **Gestión de Rutas de Modelos de Gazebo:** Actualiza dinámicamente la variable de entorno `GAZEBO_MODEL_PATH` para incluir el directorio local de mallas STL del brazo, garantizando la carga correcta de las geometrías de visualización en Gazebo Classic.
- **Secuenciación de Nodos mediante Manejo de Eventos:** Para evitar condiciones de carrera donde los controladores articulares intentan enlazarse antes de que el robot exista físicamente en la física del simulador, el script utiliza manejadores de eventos `RegisterEventHandler` y `OnProcessExit`. Esto fuerza que el controlador de posición de las articulaciones (`arm_controller`) se lance únicamente después de que el robot haya sido instanciado por el nodo `spawn_entity`.

F. Modelo Descriptor de la Estructura y Actuadores (arm.urdf.xacro)

El diseño estructural, los límites de movimiento y la configuración de hardware de control del gemelo virtual se definen jerárquicamente en un archivo descriptor Xacro, presentado en el Código 5 en el Apéndice E. Este descriptor se estructura a partir de los siguientes elementos clave:

- **Geometría de Eslabones y Mallas STL:** Vincula directamente los archivos STL exportados de CAD

(`base.stl`, `link1.stl`, etc.) escalados a metros (factor 0,001), definiendo la apariencia visual del robot en el simulador.

- **Límites y Dinámica de Articulaciones:** Modela los límites físicos angulares en radianes equivalentes a los límites de los servomotores físicos del prototipo. Adicionalmente, incluye parámetros dinámicos de fricción y amortiguamiento (`friction` y `damping`) para modelar el rozamiento de las articulaciones en Gazebo.
- **Integración con ros2_control:** Declara el elemento `<ros2_control>` que mapea las interfaces de comando y de estado (posiciones articulares y velocidades) del resolutor de física de Gazebo Classic (`gazebo_ros2_control/GazeboSystem`), permitiendo la inyección directa de señales desde el puente de comunicación HTTP. Esta declaración se complementa en la arquitectura de ROS 2 con el archivo de configuración `controllers.yaml`, el cual especifica la tasa de actualización de los bucles de control a 100 Hz y define un controlador síncrono del tipo `JointGroupPositionController` asociado a las tres articulaciones activas del robot.

G. Automatización de Configuración y Compilación (setup.sh)

Para simplificar la inicialización del entorno y garantizar la reproducibilidad de las pruebas, se programó un script de automatización en Bash. El funcionamiento y la lógica de este script se detallan en el Código 6 en el Apéndice F. Este script realiza las siguientes operaciones secuenciales:

- **Verificación del Entorno Aislado:** Comprueba la existencia del contenedor Distrobox (`ros2-humble`) y, en caso de no encontrarse activo, lo crea automáticamente instalando una base de Ubuntu 22.04.
- **Compilación Síncrona del Workspace:** Accede al contenedor, carga las variables de entorno de ROS 2 Humble y compila exclusivamente el paquete `arm_simulation` mediante el comando `colcon build`.
- **Generación de Scripts Lanzadores:** Escribe y otorga permisos de ejecución a los tres scripts rápidos (`launch_gazebo.sh`, `run_trajectory.sh` y `start_ui.sh`) en la raíz del proyecto para facilitar el uso diario.

H. Scripts Lanzadores del Entorno y Ejecutables

Para agilizar el flujo de trabajo del operador sin requerir la memorización de comandos de contenedor o de ROS 2, se desarrollaron tres scripts lanzadores. Su código fuente se presenta en el Código ?? en el Apéndice G. Cada script cumple una función específica dentro de la arquitectura distribuida:

- **Lanzamiento de Simulación (launch_gazebo.sh):** Limpia la variable de entorno `PATH` para remover rutas de Conda (previniendo conflictos de intérprete de Python con las dependencias de ROS 2), carga los entornos locales y ejecuta el launch de Gazebo.

- **Generador de Trayectorias (run_trajectory.sh):** Realiza la misma sanitización del entorno y ejecuta directamente el nodo controlador de interpolación cosenoidal.
- **Inicio de la Interfaz Gráfica (start_ui.sh):** Lanza de manera asíncrona el navegador predeterminado apuntando a `http://localhost:8080` y arranca concurrentemente el puente HTTP/ROS 2 (`web_bridge.py`) en segundo plano.

VI. SELECCIÓN DE ACTUADORES Y SENSORES

Para garantizar el éxito de la implementación física y su congruencia con el entorno de simulación, se detalla la selección de componentes de hardware:

A. Actuadores

La arquitectura cinemática de 3 GDL resulta idónea para este caso de estudio; dado que la herramienta es un haz láser unidireccional colineal con el eje de aproximación, y el objetivo es el corte y grabado sobre superficies cilíndricas, el control del posicionamiento cartesiano en los tres ejes (X, Y, Z) es crítico para compensar la curvatura de la pieza y mantener la distancia focal constante del haz. La omisión del control de orientación redundante (pitch/roll) disminuye drásticamente el torque estático requerido en las articulaciones proximales (base y hombro), lo que mitiga las deflexiones estructurales y maximiza la rigidez y repetibilidad del sistema. El diseño se complementa ubicando estratégicamente actuadores de baja masa en los eslabones distales para minimizar el tensor de inercia del mecanismo.

- **Actuador de rotación principal (Base):** Se implementó un servomotor de alto torque y engranajes metálicos (MG996R) en la base para accionar el giro del brazo a la izquierda y derecha. Este actuador soporta las cargas dinámicas y el momento de inercia de toda la estructura móvil.
- **Actuadores de elevación (Hombro y Codo):** Se utilizó un servomotor de alto torque MG996R para el hombro (articulación pegada a la base que hace elevar el brazo) para soportar el peso y los esfuerzos de flexión de los eslabones, y un servomotor de engranajes metálicos MG90S en la articulación del codo para controlar la elevación distal.
- **Efector final (Módulo Láser de estado sólido):** Consiste en un diodo láser de baja potencia con lógica transistor-transistor (TTL) de 5V, el cual es conmutado como un nivel lógico de entrada/salida de propósito general (GPIO) o vía PWM desde el ESP32. Esto permite una activación síncrona y de baja latencia acoplada al perfil cinemático de la trayectoria, emulando la dinámica de interpolación de un proceso industrial de corte láser CNC.

B. Sensores

- **Potenciómetros internos:** Los servomotores ya incluyen sensores de posición (potenciómetros) internamente. Esto mantiene el diseño mecánico limpio ya que no se requiere

acoplar encoders ópticos externos para leer los ángulos de las articulaciones.

- **Sensor de Corriente (INA219):** Se integró un sensor de corriente (ej. INA219) vía el protocolo de circuito integrado (I2C) al ESP32 para monitorear el consumo general. Sirve como mecanismo de seguridad: si el brazo choca físicamente, el pico de corriente es detectado por el ESP32, ordenando una parada de emergencia inmediata para evitar daños en las piezas impresas en 3D.

C. Especificación de Componentes

En la Tabla II se detallan las especificaciones y funciones de los componentes de hardware seleccionados.

Tabla II
ESPECIFICACIONES DE LOS COMPONENTES DE HARDWARE

Componente	Modelo	Función en el Sistema
Controlador	ESP32	Control de bajo nivel y PWM.
Actuador base	MG996R (1 ud.)	Rotación izquierda/derecha.
Actuador hombro	MG996R (1 ud.)	Elevación del hombro.
Actuador codo	MG90S (1 ud.)	Elevación del codo.
Efector final	Diodo Láser (5V)	Haz de corte de baja potencia.
Sensor corr.	INA219	Monitoreo y parada de emergencia.
Estructura	PETG	Piezas impresas en 3D.
Alimentación	Fuente 5V 5A	Energía para actuadores y láser.

D. Presupuesto de Masa y Dimensiones

En la Tabla III se detalla la masa y dimensiones físicas estimadas para los diferentes componentes del brazo manipulador.

Tabla III
PRESUPUESTO DE MASA Y DIMENSIONES DE LOS COMPONENTES

Componente	Masa (g)	Cant.	Dimensiones (mm)
ESP32 NodeMCU	10	1	48.0 × 26.0 × 12.0
Servo MG996R	55	2	40.7 × 19.7 × 42.9
Servo MG90S	13.4	1	22.8 × 12.2 × 28.5
Diodo Láser 5V	10	1	∅ 12.0 × 35.0
Sensor INA219	5	1	25.4 × 22.8 × 3.0
Estructura 3D (PETG)	350	1	Envolvente ≈ 250 × 150
Fuente 5V 5A	150	1	110.0 × 78.0 × 36.0
Cables y tornillería	50	1	N/A
Masa Total	698 g	(≈ 0.70 kg)	

E. Presupuesto y Consumo de Energía

La Tabla IV detalla el voltaje nominal y las corrientes de funcionamiento promedio y de pico para estimar la potencia del sistema.

Tabla IV
PRESUPUESTO Y CONSUMO DE ENERGÍA DE LOS COMPONENTES

Componente	Voltaje (V)	Corr. Prom.	Corr. Pico
ESP32	5.0	120 mA	240 mA
MG996R (2 ud.)	5.0	1000 mA	5.0 A
MG90S (1 ud.)	5.0	180 mA	0.8 A
Láser 5V	5.0	30 mA	30 mA
INA219	3.3	1 mA	1 mA
Consumo Promedio		1.33 A (≈ 6.7 W)	
Consumo Pico		6.07 A (≈ 30.4 W)	

F. Estimación de Costos

El presupuesto estimado para la adquisición de los componentes de hardware requeridos se presenta en la Tabla V.

Tabla V
PRESUPUESTO Y ESTIMACIÓN DE COSTOS

Descripción	Cant.	Unit. (\$/.)	Total (\$/.)
ESP32 NodeMCU	1	25.00	25.00
Servomotor MG996R	2	30.00	60.00
Servomotor MG90S	1	12.00	12.00
Módulo Láser 5V TTL	1	15.00	15.00
Sensor INA219 I2C	1	12.00	12.00
Filamento PETG (1 kg)	1	70.00	70.00
Fuente de alimentación 5V 5A	1	35.00	35.00
Cables y tornillería	-	15.00	15.00
Total Estimado			\$/ 244.00

VII. APLICACIÓN INDUSTRIAL

A diferencia de los pantógrafos y cortadoras láser cartesianas convencionales, los cuales están limitados a trazar trayectorias coplanares bidimensionales (2D) en láminas planas, la configuración antropomórfica de 3 GDL permite posicionar la herramienta en un espacio tridimensional (X, Y, Z). Esta característica habilita la aplicación del robot en procesos industriales de corte y grabado sobre superficies irregulares y no coplanares, tales como el perfilado de cuerpos curvados y el calado de piezas cilíndricas.

En la industria de manufactura mecánica, el procesamiento de piezas no planas requiere realizar cortes y grabados con perfiles tridimensionales complejos (por ejemplo, intersecciones de geometrías cilíndricas o marcas en piezas curvas). Al colocar un haz láser en el efector final del robot, es posible realizar este proceso sin contacto mecánico directo. A partir del modelo de la superficie cilíndrica de una pieza de radio R_{tubo} colocada a lo largo del eje X_0 a una altura de desfase z_{tubo} , la coordenada vertical de corte z_E se calcula dinámicamente en función de la posición transversal y_E :

$$z_E = z_{tubo} + \sqrt{R_{tubo}^2 - y_E^2} \quad (24)$$

El resolutor de cinemática inversa desarrollado en (13)-(20) calcula de manera instantánea las posiciones articulares ($\theta_1, \theta_2, \theta_3$) para que el láser siga esta trayectoria no planar, manteniendo la distancia focal constante respecto a la pared de la pieza. Esto demuestra el valor industrial del manipulador

propuesto frente a las soluciones cartesianas convencionales de bajo costo, ampliando el alcance de automatización hacia geometrías complejas no convencionales.

APÉNDICE A CÓDIGO DEL RESOLVEDOR DE CINEMÁTICA INVERSA EN LA INTERFAZ WEB (APP.JS)

```

1 function solveIK(x, y, z) {
2   const targetX = parseFloat(x);
3   const targetY = parseFloat(y);
4   const targetZ = parseFloat(z);
5   if (isNaN(targetX) || isNaN(targetY) || isNaN(targetZ)) {
6     return { error: 'VALORES_INVALIDOS', msg: 'Ingresa valores...' };
7   }
8   const theta1 = Math.atan2(targetY, targetX);
9   const j1 = theta1 * 180 / Math.PI;
10  if (j1 < jointsConfig[1].min || j1 > jointsConfig[1].max) {
11    return { error: 'FUERA_DE_ALCANCE', msg: 'Base fuera...' };
12  }
13  const xc = Math.sqrt(targetX * targetX + targetY * targetY);
14  const zc = targetZ - dl;
15  const psi_numerator = xc * xc + zc * zc - L1 * L1 - L2 * L2;
16  const psi_denominator = 2.0 * L1 * L2;
17  const psi = psi_numerator / psi_denominator;
18  if (psi < -1.0 || psi > 1.0) {
19    return { error: 'FUERA_DE_ALCANCE', msg: 'Fuera de alcance...' };
20  }
21  const solutions = [-1, 1];
22  for (let s of solutions) {
23    const sinTheta3 = s * Math.sqrt(1.0 - psi * psi);
24    const theta3 = Math.atan2(sinTheta3, psi);
25    const k1 = L1 + L2 * Math.cos(theta3);
26    const k2 = L2 * Math.sin(theta3);
27    const theta2 = Math.atan2(k1 * zc - k2 * xc, k1 * xc + k2 * zc);
28    const j2 = theta2 * 180 / Math.PI;
29    const j3 = theta3 * 180 / Math.PI;
30    if (j2 >= jointsConfig[2].min && j2 <= jointsConfig[2].max &&
31        j3 >= jointsConfig[3].min && j3 <= jointsConfig[3].max) {
32      return { j1, j2, j3 };
33    }
34  }
35  return { error: 'FUERA_DE_ALCANCE', msg: 'Inalcanzable...' };
36 }

```

Listing 1. Resolutor interactivo de cinemática inversa en JS (app.js).

APÉNDICE B CÓDIGO DEL GENERADOR DE TRAYECTORIAS DE SIMULACIÓN (TRAJECTORY_GENERATOR.PY)

```

1 def timer_callback(self):
2   if self.step_counter >= self.total_steps_for_transition:
3     self.prepare_next_transition()
4     t = float(self.step_counter) / float(self.total_steps_for_transition)
5     smooth_t = (1.0 - math.cos(t * math.pi)) / 2.0
6     current_command = []
7     for start, target in zip(self.start_pose, self.target_pose):
8       val = start + (target - start) * smooth_t
9       current_command.append(val)
10    msg = Float64MultiArray()
11    msg.data = current_command
12    self.publisher_.publish(msg)
13    self.step_counter += 1

```

Listing 2. Generación de perfiles S-curve por interpolación cosenoidal (trajectory_generator.py).

APÉNDICE C CÓDIGO DEL PUENTE DE COMUNICACIÓN ROS 2 - INTERFAZ WEB (WEB_BRIDGE.PY)

```

1 def publish_joints(self, j1_deg, j2_deg, j3_deg):
2   j1_rad = float(j1_deg) * math.pi / 180.0
3   j2_rad = float(j2_deg) * math.pi / 180.0
4   j3_rad = float(j3_deg) * math.pi / 180.0
5   msg = Float64MultiArray()
6   msg.data = [j1_rad, j2_rad, j3_rad]
7   self.publisher_.publish(msg)
8   self.get_logger().info(f'Publicado en ROS -> J1: {j1_rad:.4f} rad, J2: {j2_rad:.4f} rad, J3: {j3_rad:.4f} rad')

```

Listing 3. Conversión de unidades y envío al simulador (web_bridge.py).

APÉNDICE D

CÓDIGO DEL LANZAMIENTO DE SIMULACIÓN EN ROS 2 (GAZEBO.LAUNCH.PY)

```
1 import os
2 from ament_index_python.packages import get_package_share_directory
3 from launch import LaunchDescription
4 from launch.actions import IncludeLaunchDescription, RegisterEventHandler
5 from launch.event_handlers import OnProcessExit
6 from launch.launch_description_sources import PythonLaunchDescriptionSource
7 from launch_ros.actions import Node
8 import xacro
9
10 def generate_launch_description():
11     pkg_share = get_package_share_directory('arm_simulation')
12     xacro_file = os.path.join(pkg_share, 'urdf', 'arm.urdf.xacro')
13
14     pkg_share_parent = os.path.dirname(pkg_share)
15     default_gazebo_models = '/usr/share/gazebo-11/models:/usr/share/gazebo/models'
16     if 'GAZEBO_MODEL_PATH' in os.environ:
17         os.environ['GAZEBO_MODEL_PATH'] = pkg_share_parent + ':' +
18         default_gazebo_models + ':' + os.environ['GAZEBO_MODEL_PATH']
19     else:
20         os.environ['GAZEBO_MODEL_PATH'] = pkg_share_parent + ':' +
21         default_gazebo_models
22
23     os.environ['GAZEBO_MODEL_DATABASE_URI'] = ''
24
25     import re
26     robot_description_config = xacro.process_file(xacro_file)
27     robot_desc = robot_description_config.toxml()
28     robot_desc = re.sub(r'<!--.*?-->', '', robot_desc, flags=re.DOTALL)
29
30     robot_state_publisher_node = Node(
31         package='robot_state_publisher',
32         executable='robot_state_publisher',
33         name='robot_state_publisher',
34         output='screen',
35         parameters=[{'robot_description': robot_desc, 'use_sim_time': True}]
36     )
37
38     gazebo = IncludeLaunchDescription(
39         PythonLaunchDescriptionSource([os.path.join(
40             get_package_share_directory('gazebo_ros'), 'launch', 'gazebo.launch.py'
41         )]),
42         launch_arguments={'verbose': 'true'}.items()
43     )
44
45     spawn_entity = Node(
46         package='gazebo_ros',
47         executable='spawn_entity.py',
48         arguments=['-topic', 'robot_description', '-entity', 'arm_3gdl', '--timeout',
49             '120'],
50         output='screen'
51     )
52
53     spawn_joint_state_broadcaster = Node(
54         package='controller_manager',
55         executable='spawner',
56         arguments=['joint_state_broadcaster', '--controller-manager', '/controller_manager'],
57         output='screen'
58     )
59
60     spawn_arm_controller = Node(
61         package='controller_manager',
62         executable='spawner',
63         arguments=['arm_controller', '--controller-manager', '/controller_manager'],
64         output='screen'
65     )
66
67     return LaunchDescription([
68         gazebo,
69         robot_state_publisher_node,
70         spawn_entity,
71         RegisterEventHandler(
72             event_handler=OnProcessExit(
73                 target_action=spawn_entity,
74                 on_exit=[spawn_joint_state_broadcaster],
75             )
76         ),
77         RegisterEventHandler(
78             event_handler=OnProcessExit(
79                 target_action=spawn_joint_state_broadcaster,
80                 on_exit=[spawn_arm_controller],
81             )
82         )
83     ])
```

Listing 4. Script de lanzamiento y control secuencial en ROS 2 (gazebo.launch.py).

APÉNDICE E

ARCHIVO DE DESCRIPCIÓN ESTRUCTURAL Y ACTUADORES XACRO (ARM.URDF.XACRO)

```
1 <?xml version="1.0"?>
2 <robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="arm_3gdl">
3
4   <material name="abb_orange">
5     <color rgba="1.0 0.45 0.0 1.0"/>
6   </material>
7   <material name="abb_black">
```

```
8     <color rgba="0.12 0.12 0.12 1.0"/>
9   </material>
10   <material name="silver">
11     <color rgba="0.75 0.75 0.75 1.0"/>
12   </material>
13   <material name="graphite">
14     <color rgba="0.25 0.25 0.25 1.0"/>
15   </material>
16   <material name="brass">
17     <color rgba="0.8 0.6 0.2 1.0"/>
18   </material>
19   <material name="laser_red">
20     <color rgba="1.0 0.0 0.0 0.8"/>
21   </material>
22
23   <link name="world"/>
24   <joint name="world_to_base" type="fixed">
25     <parent link="world"/>
26     <child link="base_link"/>
27     <origin xyz="0 0 0" rpy="1.57079632679 0 0"/>
28   </joint>
29
30   <link name="base_link">
31     <inertial>
32       <mass value="1.0"/>
33       <origin xyz="0 0 0" rpy="0 0 0"/>
34       <inertia ixx="0.005" ixy="0.0" ixz="0.0" iyy="0.005" izy="0.0" izz="0.005"/>
35     </inertial>
36     <visual>
37       <origin xyz="-0.691316 -0.869483 -1.350860" rpy="0 0 0"/>
38       <geometry>
39         <mesh filename="package://arm_simulation/meshes/custom_arm/base.stl" scale="0.001 0.001 0.001"/>
40       </geometry>
41       <material name="abb_orange"/>
42     </visual>
43   </link>
44
45   <joint name="joint1" type="revolute">
46     <origin rpy="-1.57079632679 0 0" xyz="0 0.060 0"/>
47     <parent link="base_link"/>
48     <child link="link1"/>
49     <axis xyz="0 0 1"/>
50     <limit lower="-2.87979" upper="2.87979" effort="20.0" velocity="1.0"/>
51     <dynamics damping="1.0" friction="1.0"/>
52   </joint>
53
54   <link name="link1">
55     <inertial>
56       <mass value="0.5"/>
57       <origin xyz="0 0 0" rpy="0 0 0"/>
58       <inertia ixx="0.002" ixy="0.0" ixz="0.0" iyy="0.002" izy="0.0" izz="0.002"/>
59     </inertial>
60     <visual>
61       <origin xyz="-0.691316 1.350860 -0.929483" rpy="1.57079632679 0 0"/>
62       <geometry>
63         <mesh filename="package://arm_simulation/meshes/custom_arm/link1.stl" scale="0.001 0.001 0.001"/>
64       </geometry>
65       <material name="abb_orange"/>
66     </visual>
67   </link>
68
69   <joint name="joint2" type="revolute">
70     <origin rpy="1.57079632679 0 0" xyz="0.014915 0 0.036173"/>
71     <parent link="link1"/>
72     <child link="link2"/>
73     <axis xyz="0 0 1"/>
74     <limit lower="-1.91986" upper="1.91986" effort="25.0" velocity="1.0"/>
75     <dynamics damping="1.0" friction="1.0"/>
76   </joint>
77
78   <link name="link2">
79     <inertial>
80       <mass value="0.5"/>
81       <origin xyz="0 0 0" rpy="0 0 0"/>
82       <inertia ixx="0.002" ixy="0.0" ixz="0.0" iyy="0.002" izy="0.0" izz="0.002"/>
83     </inertial>
84     <visual>
85       <origin xyz="-0.706231 -0.965655 -1.350860" rpy="0 0 0"/>
86       <geometry>
87         <mesh filename="package://arm_simulation/meshes/custom_arm/link2.stl" scale="0.001 0.001 0.001"/>
88       </geometry>
89       <material name="abb_orange"/>
90     </visual>
91   </link>
92
93   <joint name="joint3" type="revolute">
94     <origin rpy="0 0 0" xyz="0.075256 0.125332 0"/>
95     <parent link="link2"/>
96     <child link="link3"/>
97     <axis xyz="0 0 1"/>
98     <limit lower="-1.91986" upper="1.22173" effort="20.0" velocity="1.5"/>
99     <dynamics damping="1.0" friction="1.0"/>
100   </joint>
101
102   <link name="link3">
103     <inertial>
104       <mass value="0.3"/>
105       <origin xyz="0 0 0" rpy="0 0 0"/>
106       <inertia ixx="0.001" ixy="0.0" ixz="0.0" iyy="0.001" izy="0.0" izz="0.001"/>
107     </inertial>
108     <visual>
109       <origin xyz="-0.781487 -1.090987 -1.350860" rpy="0 0 0"/>
110       <geometry>
111         <mesh filename="package://arm_simulation/meshes/custom_arm/link3.stl" scale="0.001 0.001 0.001"/>
112       </geometry>
113       <material name="abb_orange"/>
114     </visual>
```



```

115 </link>
116
117 <link name="flange"/>
118 <joint type="fixed" name="joint_3-flange">
119   <origin xyz="0.158 -0.030 -0.004" rpy="0 0 -0.191392"/>
120   <parent link="link3"/>
121   <child link="flange"/>
122 </joint>
123
124 <link name="tool0"/>
125 <joint name="flange-tool0" type="fixed">
126   <origin xyz="0 0 0" rpy="0 1.57079632679 0" />
127   <parent link="flange" />
128   <child link="tool0" />
129 </joint>
130
131 <gazebo>
132   <plugin name="gazebo_ros2_control" filename="libgazebo_ros2_control.so">
133     <parameters>$(find arm_simulation)/config/controllers.yaml</parameters>
134   </plugin>
135 </gazebo>
136
137 <ros2_control name="GazeboSystem" type="system">
138   <hardware>
139     <plugin>gazebo_ros2_control/GazeboSystem</plugin>
140   </hardware>
141   <joint name="joint1">
142     <command_interface name="position">
143       <param name="min">-2.87979</param>
144       <param name="max">2.87979</param>
145     </command_interface>
146     <state_interface name="position">
147       <param name="initial_value">0.0</param>
148     </state_interface>
149     <state_interface name="velocity"/>
150   </joint>
151   <joint name="joint2">
152     <command_interface name="position">
153       <param name="min">-1.91986</param>
154       <param name="max">1.91986</param>
155     </command_interface>
156     <state_interface name="position">
157       <param name="initial_value">0.0</param>
158     </state_interface>
159     <state_interface name="velocity"/>
160   </joint>
161   <joint name="joint3">
162     <command_interface name="position">
163       <param name="min">-1.91986</param>
164       <param name="max">1.22173</param>
165     </command_interface>
166     <state_interface name="position">
167       <param name="initial_value">0.0</param>
168     </state_interface>
169     <state_interface name="velocity"/>
170   </joint>
171 </ros2_control>
172
173 <xacro:macro name="visualize_frame" params="parent prefix xyz:=0 0 0' rpy:=0 0 0'
174   0' scale:=0.05">
175   <link name="${prefix}_frame_x">
176     <inertial>
177       <mass value="0.0001"/>
178       <origin xyz="0 0 0" rpy="0 0 0"/>
179       <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" iyz="0" izz="1e-9"/>
180     </inertial>
181     <visual>
182       <origin xyz="${scale/4} 0 0" rpy="0 1.57079632679 0"/>
183       <geometry>
184         <cylinder radius="${scale*0.01}" length="${scale/2}"/>
185       </geometry>
186       <material name="red">
187         <color rgba="1.0 0.0 0.0 0.8"/>
188       </material>
189     </visual>
190   </link>
191   <joint name="${prefix}_frame_x_joint" type="fixed">
192     <origin xyz="${xyz}" rpy="${rpy}"/>
193     <parent link="${parent}"/>
194     <child link="${prefix}_frame_x"/>
195   </joint>
196   <gazebo reference="${prefix}_frame_x">
197     <material>Gazebo/Red</material>
198   </gazebo>
199
200   <link name="${prefix}_frame_y">
201     <inertial>
202       <mass value="0.0001"/>
203       <origin xyz="0 0 0" rpy="0 0 0"/>
204       <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" iyz="0" izz="1e-9"/>
205     </inertial>
206     <visual>
207       <origin xyz="0 ${scale/4} 0" rpy="-1.57079632679 0 0"/>
208       <geometry>
209         <cylinder radius="${scale*0.01}" length="${scale/2}"/>
210       </geometry>
211       <material name="green">
212         <color rgba="0.0 1.0 0.0 0.8"/>
213       </material>
214     </visual>
215   </link>
216   <joint name="${prefix}_frame_y_joint" type="fixed">
217     <origin xyz="${xyz}" rpy="${rpy}"/>
218     <parent link="${parent}"/>
219     <child link="${prefix}_frame_y"/>
220   </joint>
221   <gazebo reference="${prefix}_frame_y">
222     <material>Gazebo/Green</material>
223   </gazebo>
224
225   <link name="${prefix}_frame_z">

```

```

225     <inertial>
226       <mass value="0.0001"/>
227       <origin xyz="0 0 0" rpy="0 0 0"/>
228       <inertia ixx="1e-9" ixy="0" ixz="0" iyy="1e-9" iyz="0" izz="1e-9"/>
229     </inertial>
230     <visual>
231       <origin xyz="0 0 ${scale/4}" rpy="0 0 0"/>
232       <geometry>
233         <cylinder radius="${scale*0.01}" length="${scale/2}"/>
234       </geometry>
235       <material name="blue">
236         <color rgba="0.0 0.0 1.0 0.8"/>
237       </material>
238     </visual>
239   </link>
240   <joint name="${prefix}_frame_z_joint" type="fixed">
241     <origin xyz="${xyz}" rpy="${rpy}"/>
242     <parent link="${parent}"/>
243     <child link="${prefix}_frame_z"/>
244   </joint>
245   <gazebo reference="${prefix}_frame_z">
246     <material>Gazebo/Blue</material>
247   </gazebo>
248 </xacro:macro>
249
250 <xacro:visualize_frame parent="base_link" prefix="system0" xyz="0 0 0" rpy="
251   -1.57079632679 0 0" scale="0.08"/>
252 <xacro:visualize_frame parent="link1" prefix="system1" xyz="0 0 0" rpy="0 0 0"
253   scale="0.06"/>
254 <xacro:visualize_frame parent="link2" prefix="system2" xyz="0 0 0" rpy="0 0
255   1.030041" scale="0.06"/>
256 <xacro:visualize_frame parent="link3" prefix="system3" xyz="0 0 0" rpy="0 0
257   -0.191392" scale="0.06"/>
258 <xacro:visualize_frame parent="flange" prefix="system4" xyz="0 0 0" rpy="0 0 0"
259   scale="0.06"/>
260 </robot>

```

Listing 5. Modelado estructural, cinemático y configuración de ros2_control en Xacro (arm.urdf.xacro).

APÉNDICE F

SCRIPT DE AUTOMATIZACIÓN DE CONFIGURACIÓN Y COMPILACIÓN (SETUP.SH)

```

1 #!/bin/bash
2 set -e
3 GREEN='\033[0;32m'
4 BLUE='\033[0;34m'
5 YELLOW='\033[0;33m'
6 RED='\033[0;31m'
7 NC='\033[0m'
8
9 echo -e "${BLUE}===== ${NC}"
10 echo -e "${GREEN}      KiraOne - SETUP & COMPILATION SYSTEM      ${NC}"
11 echo -e "${BLUE}===== ${NC}"
12
13 if ! command -v distrobox && /dev/null; then
14   echo -e "${RED}Error: distrobox no esta instalado. ${NC}"
15   exit 1
16 fi
17
18 echo -e "${BLUE}[1/4] Verificando contenedor Distrobox... ${NC}"
19 if ! distrobox list | grep -q "ros2-humble"; then
20   distrobox create -n ros2-humble -i ubuntu:22.04 -y
21 fi
22 echo -e "${GREEN}[OK] Contenedor 'ros2-humble' detectado. ${NC}"
23
24 echo -e "${BLUE}[2/4] Compilando Workspace de ROS2... ${NC}"
25 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
26
27 distrobox enter ros2-humble -- bash -c "
28   cd '$SCRIPT_DIR/ros2_ws' && \
29   source /opt/ros/humble/setup.bash && \
30   colcon build --packages-select arm_simulation
31 "
32 echo -e "${GREEN}[OK] Workspace compilado exitosamente. ${NC}"
33
34 echo -e "${BLUE}[3/4] Configurando permisos de ejecucion... ${NC}"
35 chmod +x "$SCRIPT_DIR/ros2_ws/src/arm_simulation/scripts/trajectory_generator.py"
36 echo -e "${GREEN}[OK] Permisos configurados. ${NC}"
37
38 echo -e "${BLUE}[4/4] Generando scripts de ejecucion rapida... ${NC}"
39 # Generacion automatica de launch_gazebo.sh, run_trajectory.sh y start_ui.sh ...

```

Listing 6. Script de inicialización y compilación del entorno (setup.sh).

APÉNDICE G

LANZADORES RÁPIDOS DE LA SIMULACIÓN Y DE LA INTERFAZ WEB

```

1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
3 echo "Iniciando Gazebo Classic (KiraOne)..."
4 distrobox enter ros2-humble -- bash -c "
5   export PATH=$(echo $PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
6   source /opt/ros/humble/setup.bash
7   source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
8   ros2 launch arm_simulation gazebo.launch.py
9 "

```

Listing 7. Script lanzador de Gazebo Classic (launch_gazebo.sh).

```

1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
3 echo "Ejecutando generador de trayectoria (KiraOne)..."
4 distrobox enter ros2-humble -- bash -c "
5     export PATH=$(echo $PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
6     source /opt/ros/humble/setup.bash
7     source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
8     /usr/bin/python3 '$SCRIPT_DIR/ros2_ws/src/arm_simulation/scripts/
9     trajectory_generator.py'

```

Listing 8. Script ejecutor de la trayectoria cosenoidal (run_trajectory.sh).

```

1 #!/bin/bash
2 SCRIPT_DIR="$( cd "$( dirname "${BASH_SOURCE[0]}" )" && /dev/null && pwd )"
3 echo "Iniciando servidor web y puente ROS 2 para KiraOne en http://localhost
4     :8080..."
5
6 if command -v xdg-open && /dev/null; then
7     (sleep 1.5 && xdg-open "http://localhost:8080") &
8 elif command -v sensible-browser && /dev/null; then
9     (sleep 1.5 && sensible-browser "http://localhost:8080") &
10 fi
11
12 distrobox enter ros2-humble -- bash -c "
13     export PATH=$(echo $PATH | tr ':' '\n' | grep -v miniconda | tr '\n' ':')
14     source /opt/ros/humble/setup.bash
15     source '$SCRIPT_DIR/ros2_ws/install/setup.bash'
16     python3 '$SCRIPT_DIR/ui/web_bridge.py'

```

Listing 9. Script de inicio del servidor y de la interfaz de control (start_ui.sh).

REFERENCIAS

- [1] J. Craig, *Introduction to Robotics: Mechanics and Control*, 4th ed. Pearson, 2017.
- [2] I. Hinojosa, “Repositorio de Código del Proyecto FP_Robotics”, GitHub, 2026. Disponible en: https://github.com/Reve7339/FP_Robotics.